Machine Learning



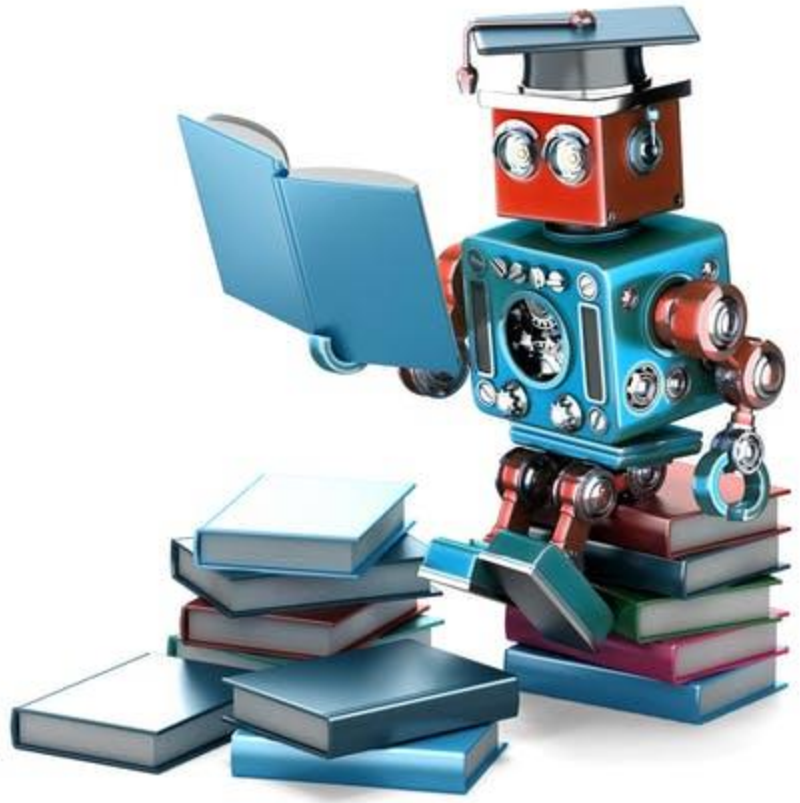
Professor Lasse Riis Østergaard, AAU
Dept. of Health Science and Technology
Medical Informatics and Image Analysis



AALBORG UNIVERSITY
DENMARK

Agenda

- Basic Classification
  - Binary and multiclass classification
  - Performance measures
- Opgaver



MNIST

Modified National Institute of Standards and Technology database

- 70.000 small images of digits
- Handwritten by high school students and employees of the US Census Bureau
- Each image is labeled with the digit it represents
- The “hello world” of machine learning
- 28 x 28 pixels = 784 features
- One pixels intensity 0 (white) to 255 (black)

- Reshape to 28 x 28 array/vector
- Training set is already shuffled



Figure 3-2. Digits from the MNIST dataset

Training a Binary Classifier

- Identify one digit – “5-detector”
- Two classes: 5 and non-5
- Classifier: SGDClassifier
- Train on the whole training set

```
>>> X, y = mnist.data, mnist.target
```

```
>>> X.shape
(70000, 784)
```

```
>>> y.shape
(70000,)
```

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]

y_train_5 = (y_train == '5')  # True for all 5s, False for all other digits
y_test_5 = (y_test == '5')

from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

```
>>> sgd_clf.predict([some_digit])
array([ True])
```

```
>>> y[0]
'5'
```

AALBORG UNIVERSITY
DENMARK

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()
```



Figure 3-1. Example of an MNIST image

Performance measures

Accuracy

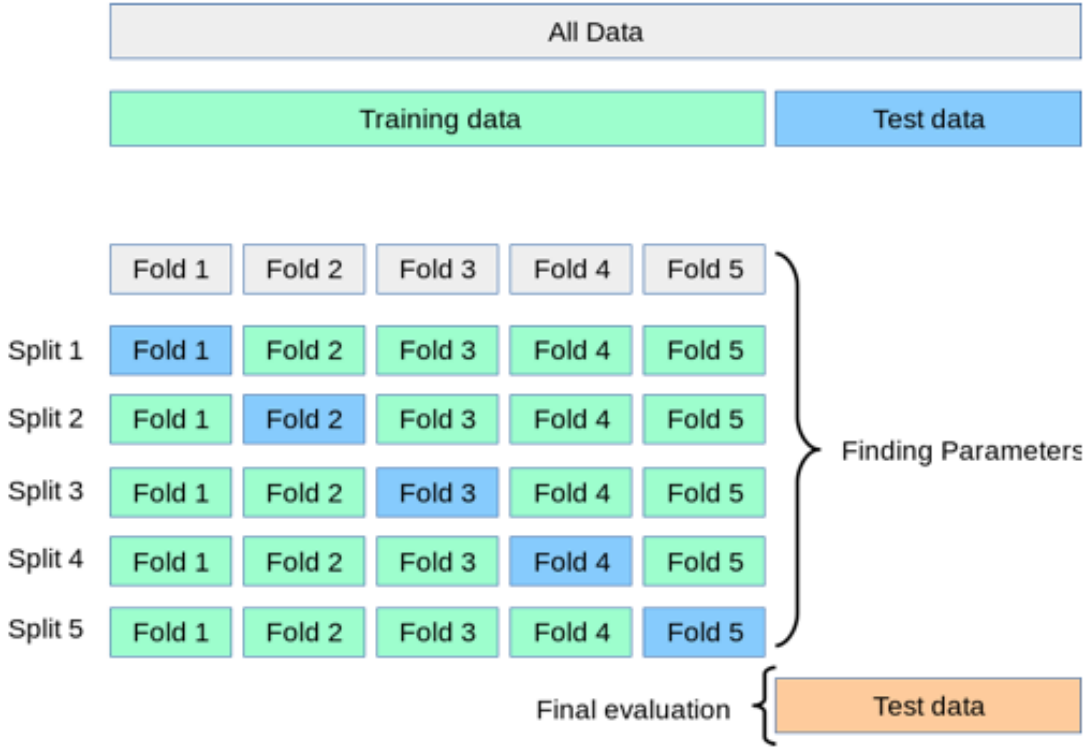
- Measuring Accuracy using Cross-Validation

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train)))  # prints False: no 5s detected
```

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```



Confusion Matrices

- Each column of the matrix represents the instances in a predicted class while each row represents the instances in an actual class, or vice versa

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,   687],
       [ 1891,  3530]])
```

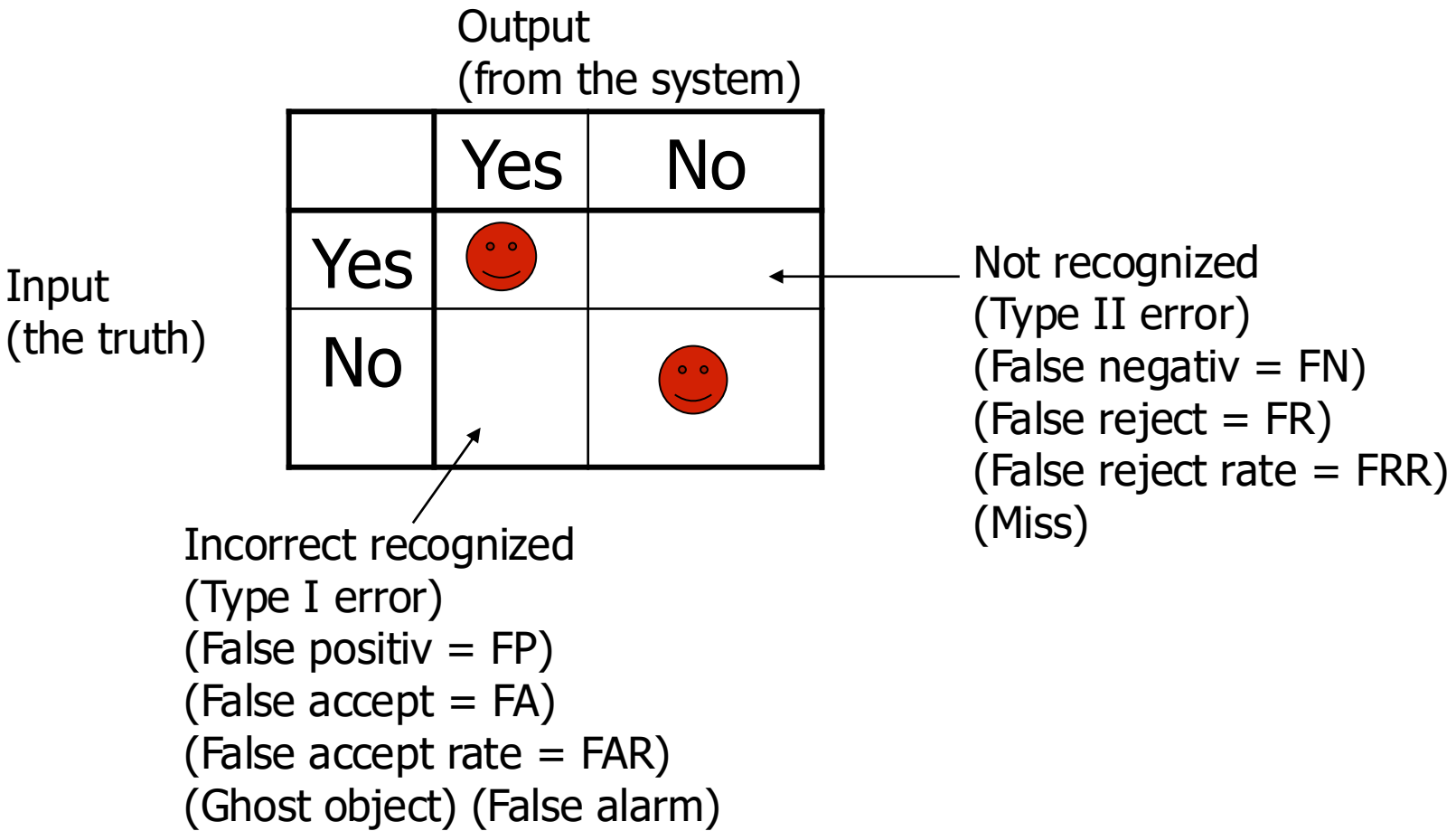
		Predicted	
		Negative **(N)** -	Positive **(P)** +
Actual	Negative -	True Negative **(TN)**	**False Positive (FP) Type I Error**
	Positive +	**False Negative (FN) Type II Error**	True Positive **(TP)**



AALBORG UNIVERSITY
DENMARK

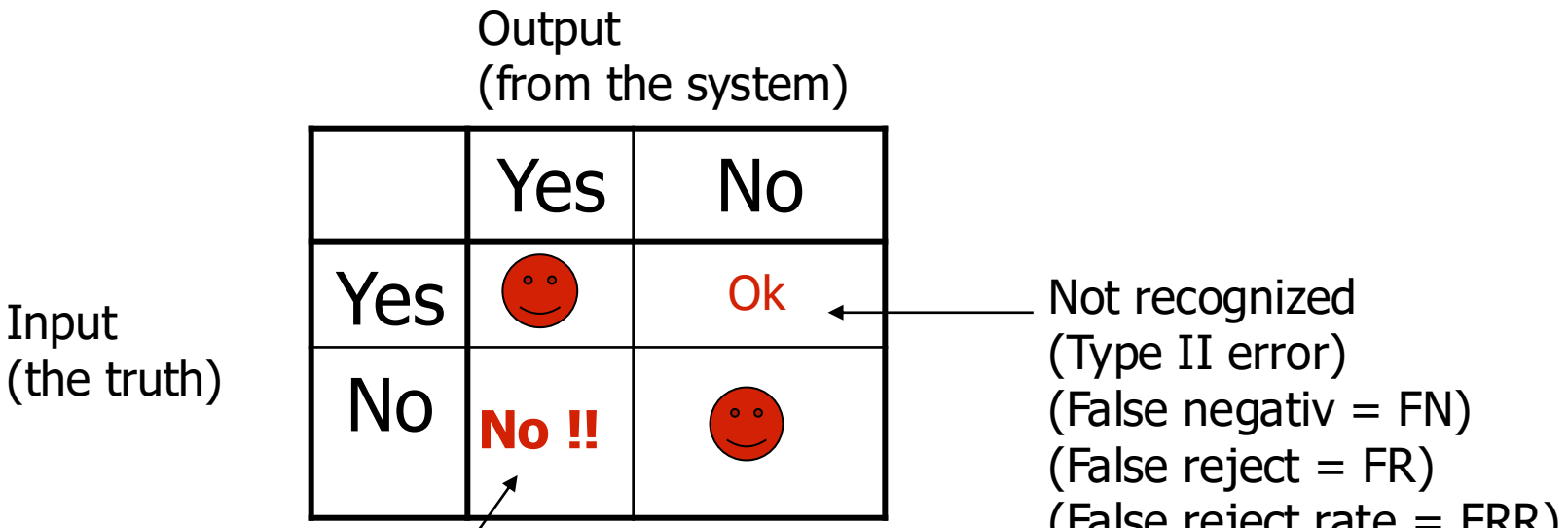
General Representation of errors

- Number of errors = Incorrect recognized + Not recognized
- The total number of errors can be represented like this:



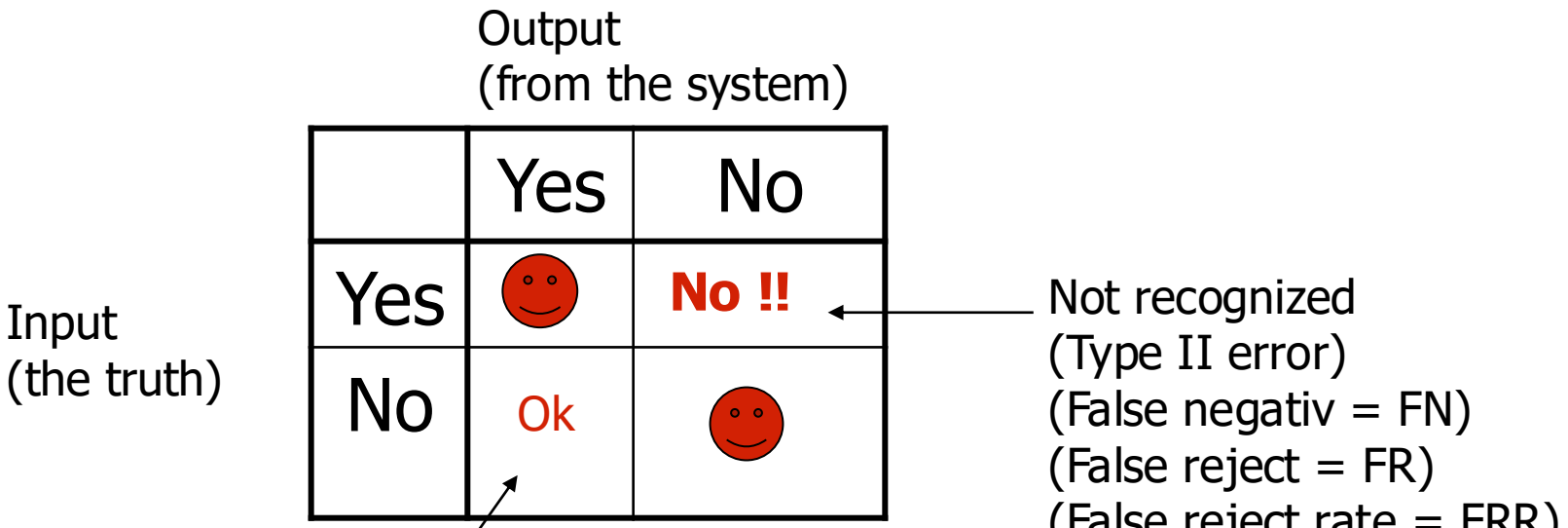
General Representation of errors

- Example: Videos
  - Detect videos that are safe for kids
  - FN versus FP – are they equally important?

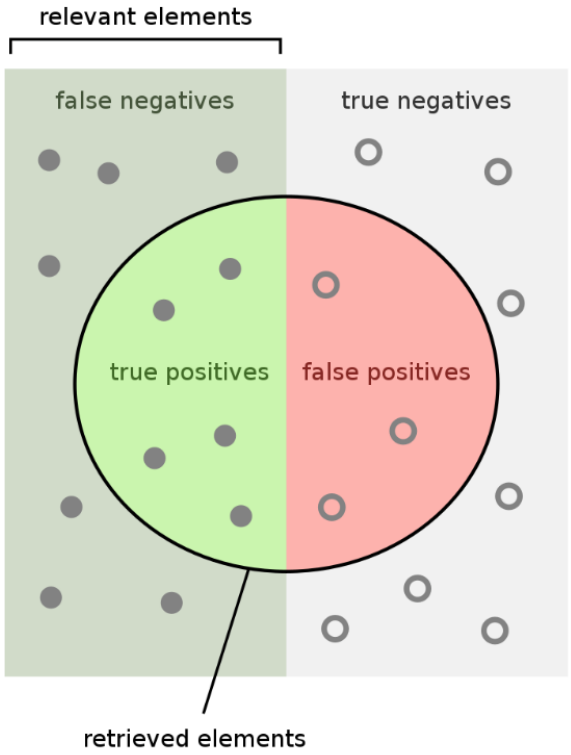


General Representation of errors

- Example: Airport security
  - Airport security screening for prohibited weapons
  - FN versus FP – are they equally important?



Precision and Recall

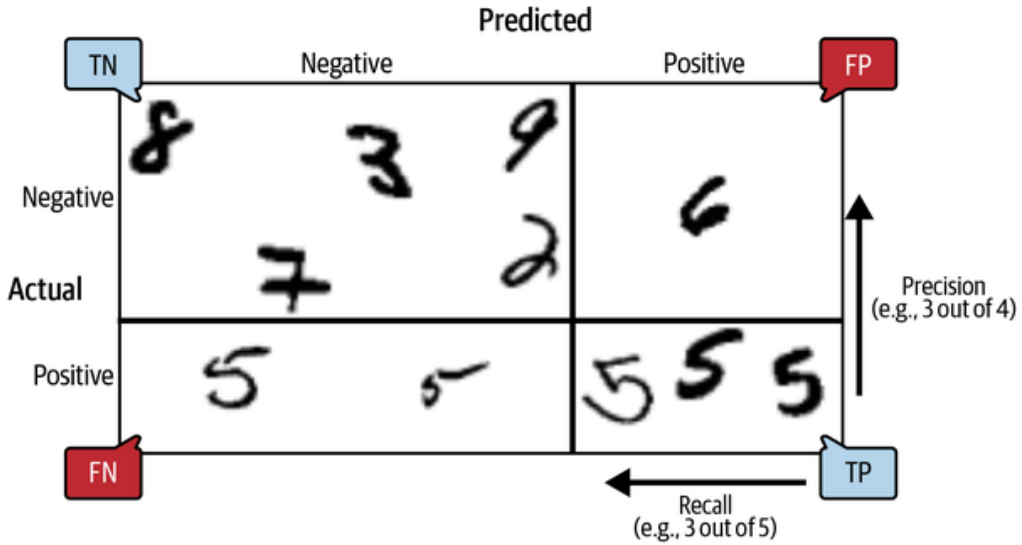


How many retrieved items are relevant?

Precision =

How many relevant items are retrieved?

Recall =




```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred)  # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred)  # == 3530 / (1891 + 3530)
0.6511713705958311
```

AALBORG UNIVERSITY
DENMARK

F1 score

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN+FP}{2}}$$

To compute the F_1 score, simply call the `f1_score()` function:

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```



AALBORG UNIVERSITY
DENMARK

The Precision/Recall Trade-off

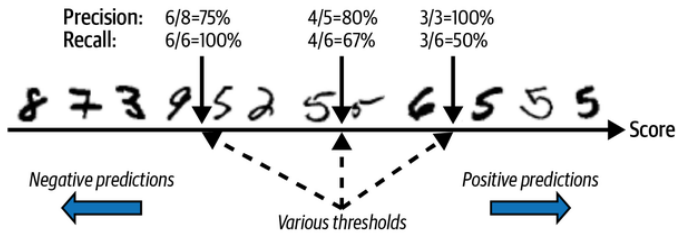


Figure 3-4. The precision/recall trade-off: images are ranked by their classifier score, and those above the chosen decision threshold are considered positive; the higher the threshold, the lower the recall, but (in general) the higher the precision

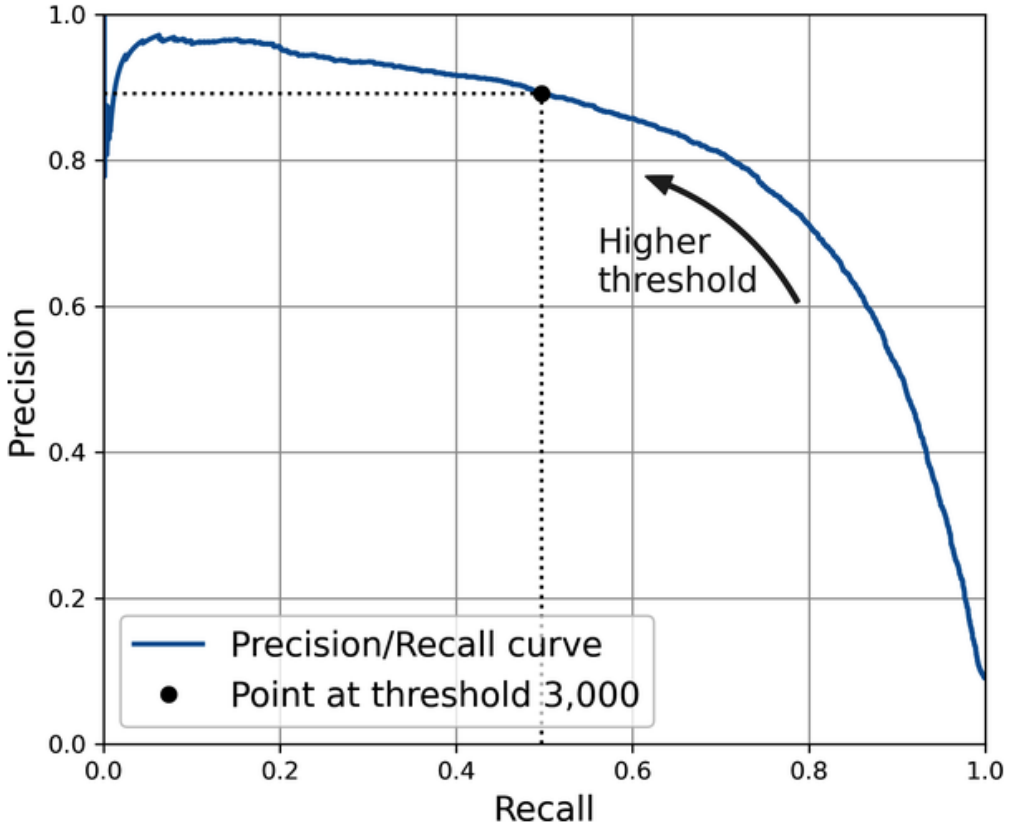


Figure 3-6. Precision versus recall

Classifier validation

Binary classification:

$$y = \begin{cases} 1 \\ \mathbf{1} \\ 0 \\ 1 \\ \mathbf{0} \\ \vdots \\ 1 \end{cases} \qquad \hat{y} = \begin{cases} 1 \\ \mathbf{0} \\ 0 \\ 1 \\ \mathbf{1} \\ \vdots \\ 1 \end{cases}$$

Confusion matrix:

	Actual YES	Actual NO
Predicted YES	TP	FP
Predicted NO	FN	TN

n = 200	Actual YES	Actual NO
Predicted YES	80	30
Predicted NO	20	70

$$\text{Sensitivity (recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{80}{80 + 20} = 80\%$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{70}{70 + 30} = 70\%$$

$$\text{PPV (precision)} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{80}{80 + 30} = 73\%$$

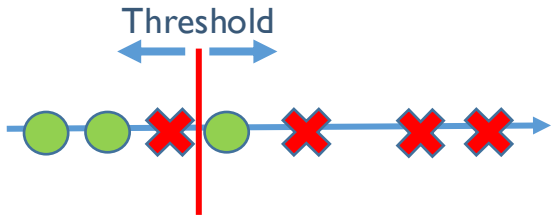
$$\text{NPV} = \frac{\text{TN}}{\text{TN} + \text{FN}} = \frac{70}{70 + 20} = 78\%$$

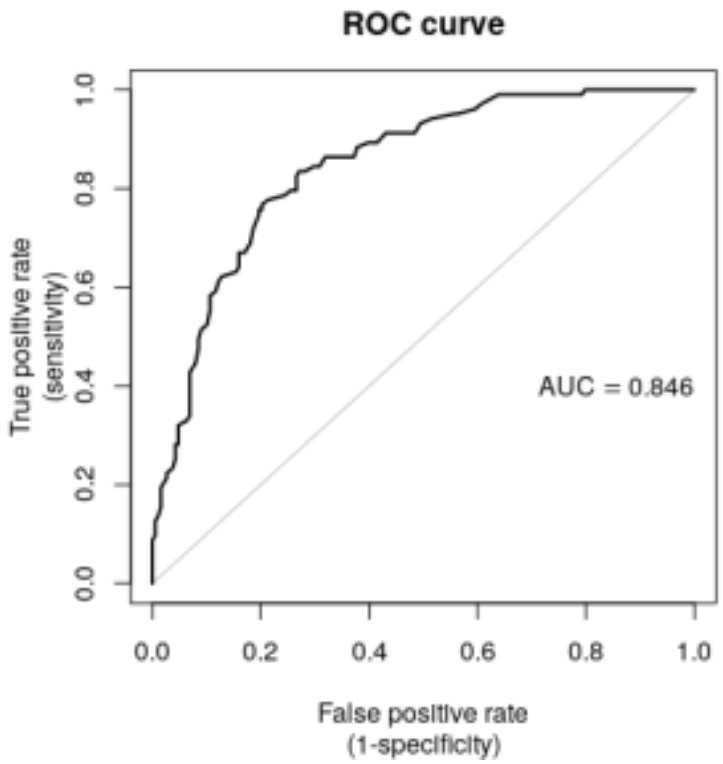
$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} = \frac{80 + 70}{200} = 75\%$$

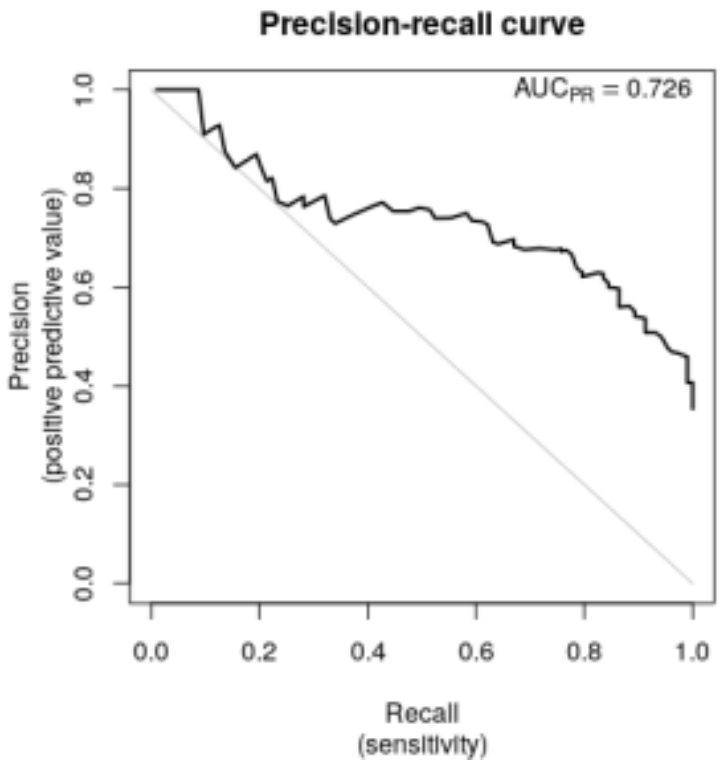
AD NYE VEJE
AALBORG UNIVERSITET

Classifier validation

Binary classification: **Varying threshold**







(Careful with unbalanced data)

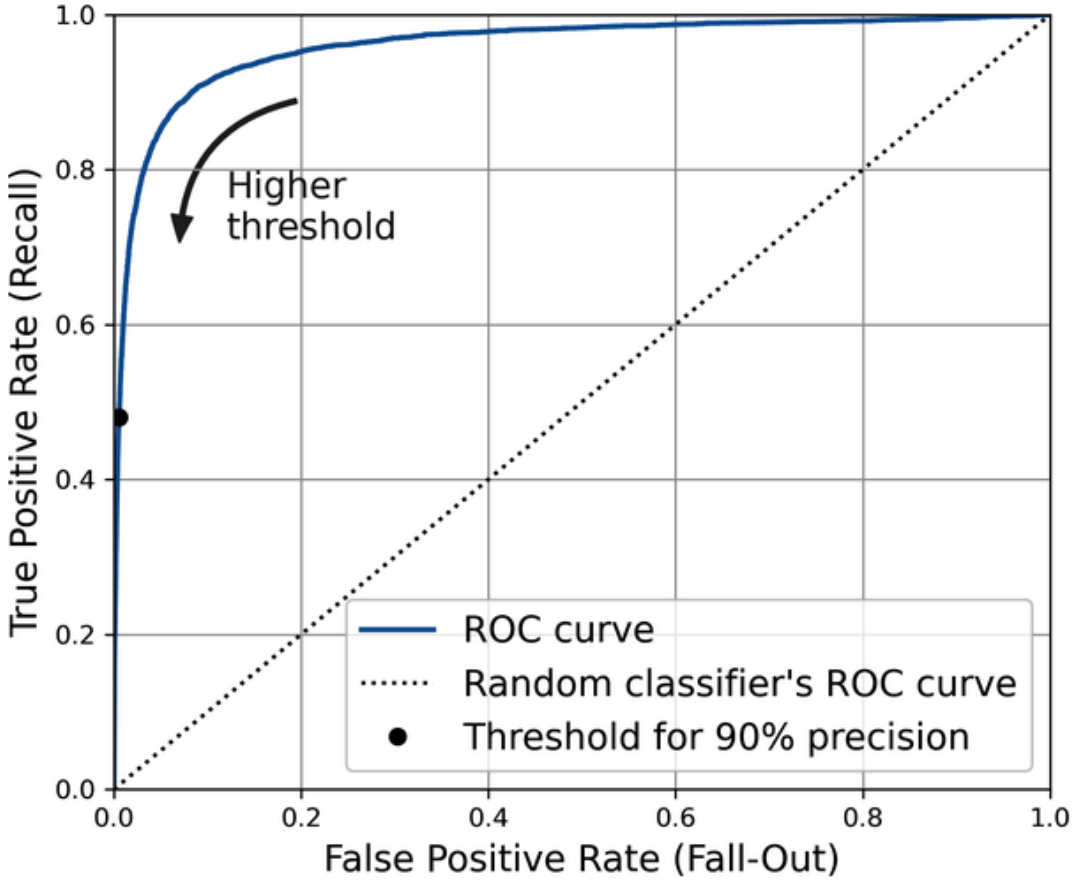
The ROC Curve

```
from sklearn.metrics import roc_curve

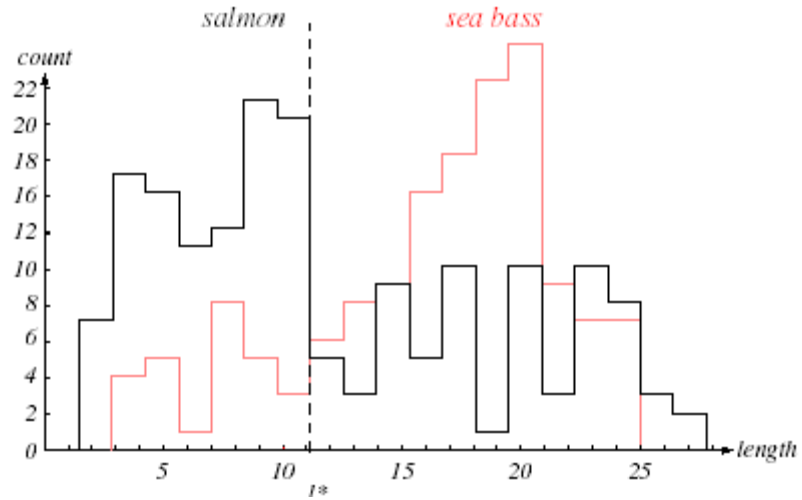
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

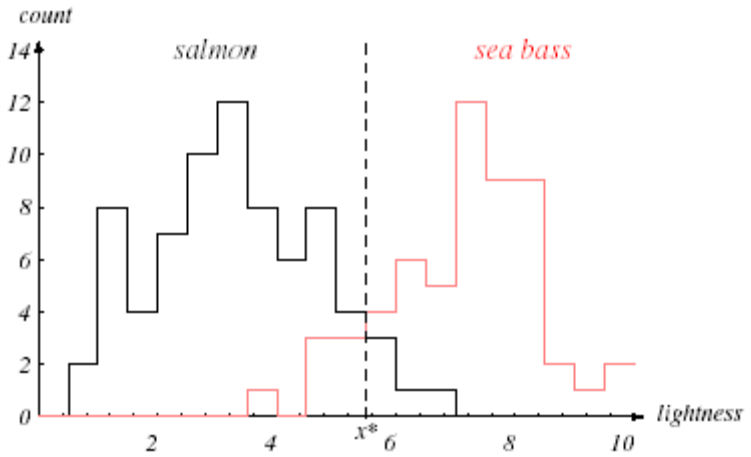
```
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")
```

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9604938554008616
```



Select Threshold





Multiclass Classification

- Multiclass classifiers can distinguish between more than two classes
- One-versus-the-rest/One-versus-all
- One-versus-one

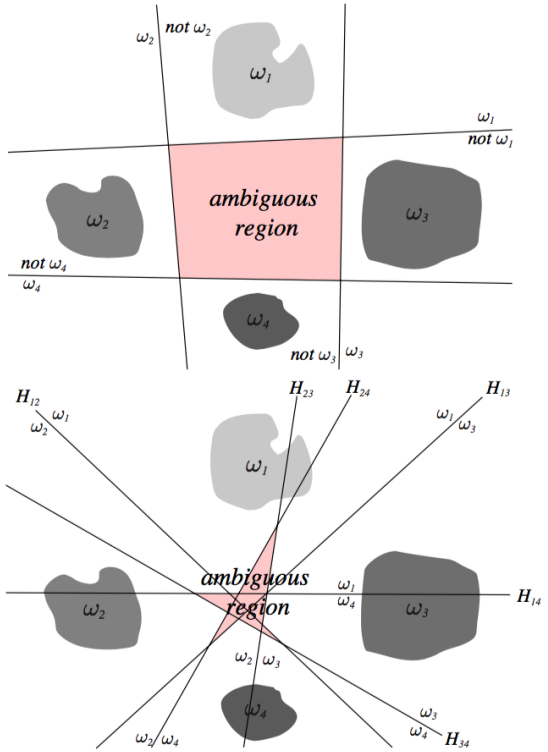


FIGURE 5.3. Linear decision boundaries for a four-class problem. The top figure shows ω_i/not ω_i dichotomies while the bottom figure shows ω_i/ω_j dichotomies and the corresponding decision boundaries H_{ij}. The pink regions have ambiguous category assignments. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

Error Analysis

Confusion matrix

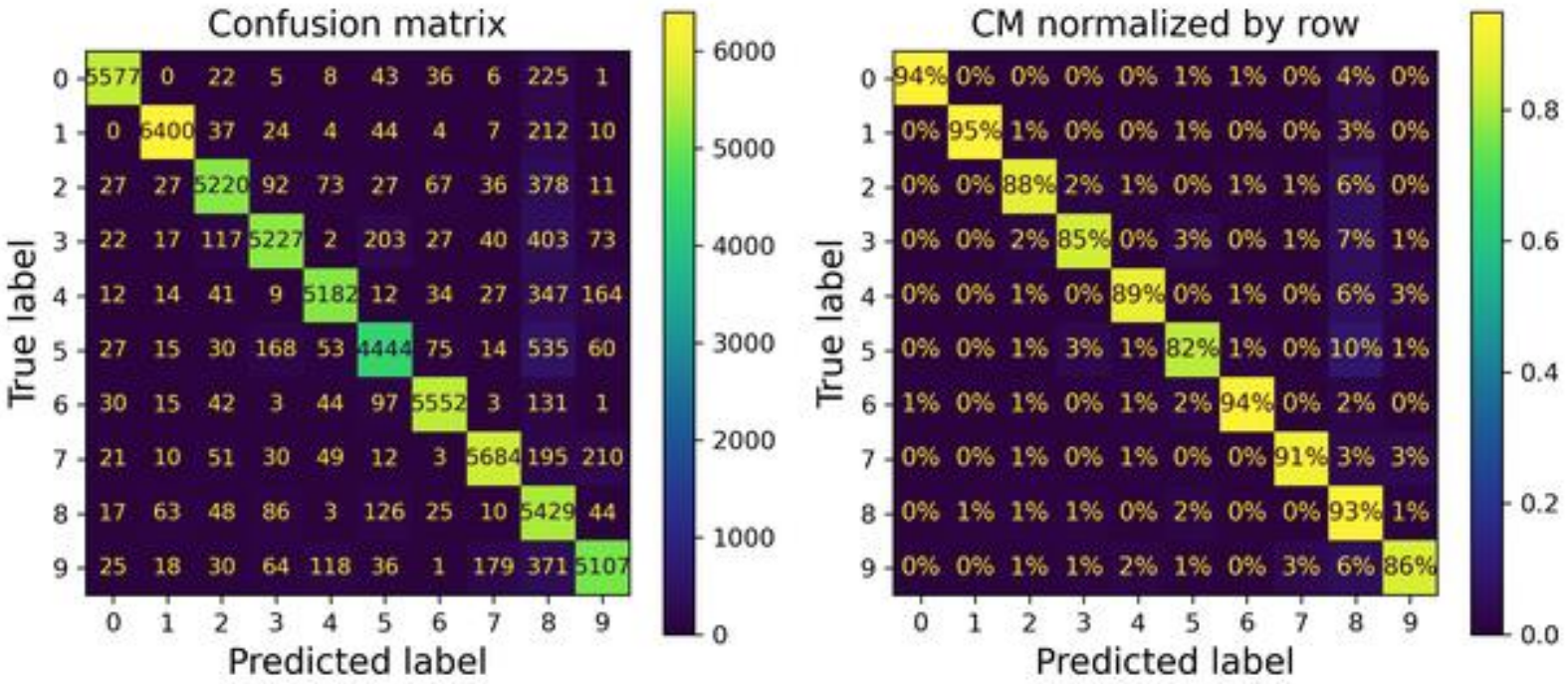


Figure 3-9. Confusion matrix (left) and the same CM normalized by row (right)

Error Analysis

Errors

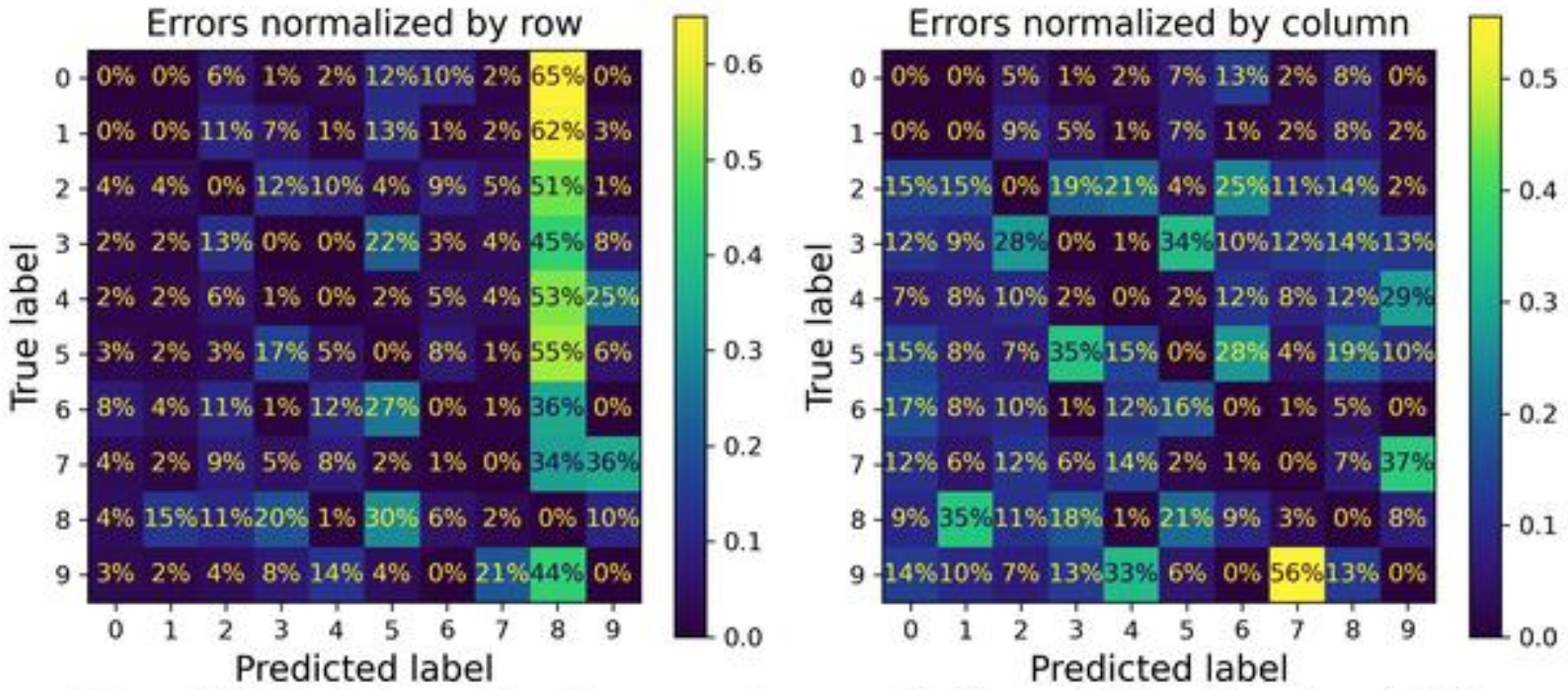
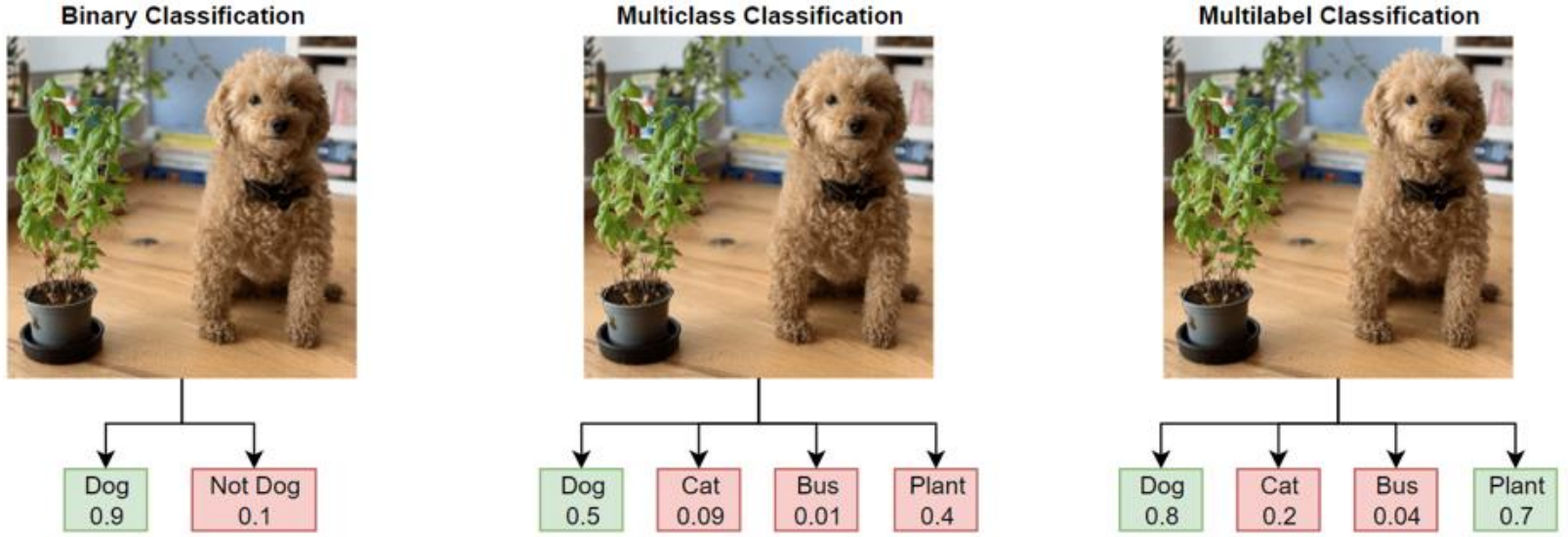


Figure 3-10. Confusion matrix with errors only, normalized by row (left) and by column (right)

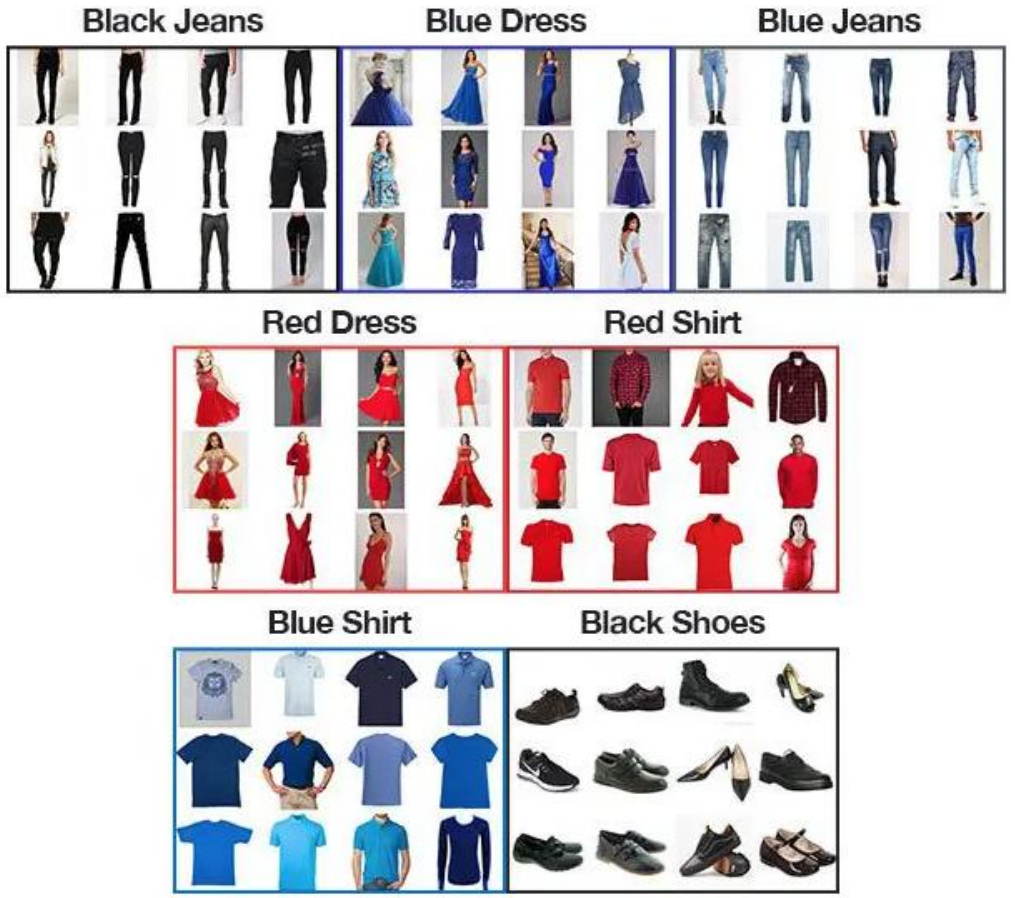
Multilabel Classification

- Output multiple classes for each instance



Multioutput Classification

- Generalization of multilabel classification



Multilabel or multioutput?



